

IMPLEMENTATION DETAILS & SNIPPETS

Developer: [Left Blank] | OptiCrop Project Documentation

Repository: <https://github.com/goutham-05-raj/OptiCrop-Smart-Agricultural-Production-Optimization-Engine>

1. Backend Routing Code (Flask)

The controller logic utilizes Flask routes to accept POST data, process it, and render the results template. Below is an excerpt of the prediction pipeline:

- Extract soil parameters from the HTML form request.
- Transform the input list using the StandardScaler pipeline.
- Perform inference using the Random Forest classifier.
- Predict estimated yield and categorise soil with the KMeans model.
- Append details to datasets/user_predictions.csv and render results.

/predict [POST, app.py]

2. Model Training Script (train_models.py)

1. Parse & Validate form fields (bounds check)

2. StandardScaler transform

3. RF predict -> crop label

4. RF regress -> yield (t/ha)

5. K-Means predict -> soil cluster

6. NPK delta -> fertiliser advice

7. Append row to user_predictions.csv

8. Render result Jinja2 template

train_models.py

Scales, encodes, trains 4 classifiers, selects best by accuracy,
dumps pipelines to models/ directory.